Doodle Classification with Classical Models and Neural Networks

Reproduction Experiments and Live Canvas Robustness

ARJUN SRIKANTH PES1UG24AM920

NIKHIL JOSHI PES1UG24AM441

Machine Learning Mini Project | 22 October 2026

Abstract

We study closed-set recognition of hand-drawn doodles using logistic regression, four support-vector-machine kernels, four compact convolutional networks, transfer learning and gradient-boosted trees. The project extends a published doodle-classification study with tuned XGBoost and a local drawing interface. Recorded experiments use 3, 10 and 50 categories with fixed class-balanced splits. The 50-class CNN V1 achieves 81.21% validation accuracy, compared with 77.95% for the final XGBoost model. Independent Mac inference confirms 81.071% and 77.886% test accuracy, respectively. A separate canvas investigation shows that rendering differences can severely reduce performance despite valid model loading. Stroke and geometry adaptation improves controlled vector-rendering diagnostics, but worsens an independent low-resolution bitmap control. We therefore expose adaptation as an optional input mode. These results support a compact CNN as an effective practical classifier and show why dataset accuracy alone is insufficient evidence for live drawing reliability.

1 Introduction

A doodle contains sparse strokes rather than the texture and shading of a photograph. Recognizing its intended object is useful when a user can draw an idea more easily than describe it. Our task is to map one drawing to one category in a fixed vocabulary. It does not retrieve photographs, search the internet or recognize arbitrary objects.

The reference study by Phu, Xiao and Muindi [1] compares classical and neural models for this task. We examine an implementation of these comparisons, add a tree-based model, and evaluate the transition from saved bitmap examples to a live canvas. The central questions are whether spatial image models remain preferable as the vocabulary grows, and whether their measured performance survives changes in input rendering.

1 1 Implementation provenance

The baseline implementation, original NVIDIA training runs and model artifacts were supplied as project materials [3]. This report documents the assembled project, its XGBoost extension, independent CPU verification and subsequent canvas analysis. Reused code and historical runs are credited explicitly; author-specific training contributions are not inferred from artifact ownership.

2 Data and evaluation

Google QuickDraw supplies labeled bitmap and vector drawings [2]. The model input is a 28 by 28 grayscale image, flattened to 784 features for classical models. White strokes lie on a black background, with uint8 values from 0 to 255. The experiment code selects the first 20,000 records per class and builds a fixed 80/10/10 split using seed 42 in two successive class-wise splits. All compared models reuse those saved arrays.

Table 1 Dataset sizes for the main experiments

Classes	Train	Validation	Test
3	48,000	6,000	6,000
10	160,000	20,000	20,000
50	800,000	100,000	100,000

The 3-class vocabulary is eraser, belt and banana. The 10-class set contains laptop, rain, hockey stick, wristwatch, nose, pond, The Mona Lisa, banana, panda and paint can. The 50-class set contains these categories and additional visually similar objects. Category ordering is shared by data archives and model outputs.

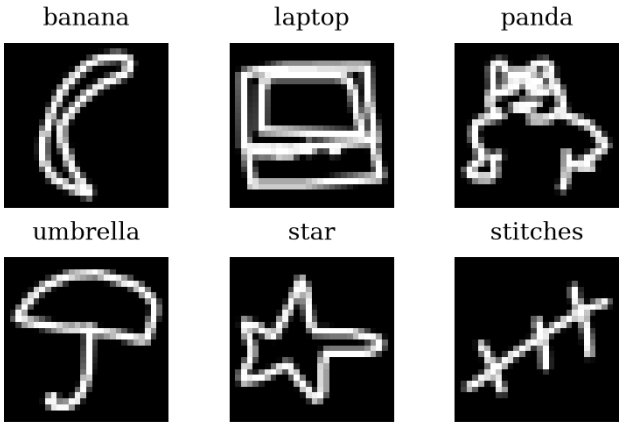


Figure 1 Six genuine examples from the saved test archive. Labels are dataset annotations; drawings are not generated illustrations.

2 1 Metrics and evidence

Accuracy is the percentage of correct top-1 predictions. Validation accuracy supports reproduction comparisons and parameter selection; test accuracy is reported separately when measured. Missing test values are not treated as measured results. Training time is the selected model fit time, not total tuning cost. Original logs, saved checkpoints and later Mac checks are distinguished throughout.

3 Methods

3 1 Logistic regression

The baseline is multinomial logistic regression on raw pixels. For class k, the linear score is transformed by a softmax: $p(y=k \mid x) = \exp(z_k) / \Sigma_j \exp(z_j)$, where $z_k = w_k^T x + b_k$. The implementation uses L-BFGS, random state 0 and a maximum of 100 iterations. This intentionally limited baseline can stop before numerical convergence; the cap is a reproduction setting, not a runtime failure.

3 2 Support vector machines

SVC models are evaluated with linear, polynomial, radial-basis-function and sigmoid kernels. A MinMaxScaler fitted only on training pixels maps features to [-1, 1]. Every run keeps C=1 and max_iter=500. Polynomial degree is 5 with coef0=1; the RBF gamma is 1; the sigmoid coef0 is 1. SVM experiments cover 3 and 10 classes. A separate data-size investigation repeats them at 70,000 records per class while retaining the iteration limit.

3 3 Convolutional networks

The CNNs learn local stroke patterns rather than independent pixel thresholds. Each begins with 14 filters of size 3 by 3 and ReLU activation. V1 contains two convolution/pooling stages and a 128-unit dense layer. Successive variants remove layers or reduce dense capacity. Table 2 gives the exact variants implemented in the current code.

Table 2 CNN architectures before the K-class output

Version	Feature extractor	Dense
V1	Conv14 - pool - Conv14 - pool	128
V2	Conv14 - pool	128
V3	Conv14	128
V4	Conv14 - pool	64

Convolutions use valid padding; pooling uses a 2 by 2 window. The final layer is a K-class softmax. Training uses Adam at learning rate 0.001, batch size 500, sparse categorical cross-entropy and 100 epochs. Seed 7 is retained for recorded CNN runs. The loss is $L = -(1/N) \Sigma_i \log p(y_i \mid x_i)$. A collapse check can stop near-chance runs after five epochs and retry with a new seed.

V1 and V2 use both raw and binarized inputs; V3 and V4 use binarized inputs. Binarization maps every nonzero pixel to 255. Inputs otherwise retain the raw 0-255 range to preserve the supplied reproduction settings. Six configurations at each of three category counts produce 18 CNN runs.

3 4 Transfer learning

Four ImageNet-initialized networks are used on three classes: VGG19, MobileNet V1, ResNet50 and InceptionV3. Grayscale images are repeated across three channels and resized to 224 by 224, or 299 by 299 for Inception. The reproduction code keeps pixel values in [0,255] without application-specific preprocess_input. This is a documented experimental choice, not a recommendation for general transfer learning.

Frozen-network features are computed once, then a new Dense128-ReLU and Dense3-softmax head is trained on globally pooled features. VGG19 uses 20 head epochs; MobileNet and ResNet50 use 100. Inception uses 50 head epochs followed by 50 fine-tuning epochs on its layers at index 249 and above, using cached mixed8 features. Head training uses Adam with clipnorm=1; stage two uses SGD at 0.0001 with momentum 0.9. Batch size is 50.

3 5 XGBoost extension

XGBoost [4] supplies a model family absent from the reference comparison. It combines additive decision trees on the same 784 raw pixels. The multi-class objective uses soft probabilities and histogram-based tree construction. Unlike a CNN, the model has no built-in spatial weight sharing. A regularized boosting objective can be written as $L = \Sigma_i \ell(y_i, \Sigma_t f_t(x_i)) + \Sigma_t \Omega(f_t)$, where f_t is a tree and Ω penalizes tree complexity.

Optuna TPE [5] searches seven settings using validation accuracy. A trial retains the boosting round with the best validation log-loss; early stopping has patience 50. Depth is capped at 9 because deeper 50-class trials exceeded the recorded 8 GB GPU memory budget. Original searches used the GPU; Mac verification uses CPU inference on the same saved models.

Table 3 Initial XGBoost search ranges

Parameter	Range
max_depth	4 to 9
learning_rate	0.02 to 0.30, log
min_child_weight	1 to 10
subsample	0.6 to 1.0
colsample_bytree	0.3 to 1.0
gamma	0 to 5
reg_lambda	0.1 to 10, log

The wider 10-class search changes depth to 6-9, min_child_weight to 1-50 and reg_lambda to 1-100. The final 50-class continuation retains its selected setting and trains beyond the initial time cap. Selection between settings uses validation results; early and extended phases both have recorded test scores, so the project is not described as using the test set only once globally.

4 Experimental results

The retained ledger contains 46 non-smoke records: 3 logistic-regression, 8 SVM, 18 CNN, 4 transfer-learning, 8 additional SVM data-size checks and 5 XGBoost records. These are configurations and follow-up runs, not 46 independent random seeds. Original experiments record an RTX 4070 Laptop GPU and Intel i7-14650HX CPU. Tables below use unrounded CSV values displayed to two decimals.

Table 4 Best validation accuracy within each family

Family	3 classes	10 classes	50 classes
LR	79.27	65.79	43.31
SVM	84.00	74.62	Not run
CNN	91.72	88.77	81.21
Transfer	95.35	Not run	Not run
XGBoost	91.65	89.84	77.95

Performance decreases as the vocabulary grows. At 50 classes, the V1 CNN leads the compared configurations at 81.21% validation accuracy; tuned XGBoost reaches 77.95%. On three classes, the transfer models are strongest. VGG19 and ResNet50 tie at 95.35% validation accuracy. These statements concern the tested configurations, not universal model rankings.

4 1 Classical results

Table 5 Classical validation accuracy and fit time

Model	K	Val %	Fit s
LR	3	79.27	19.9
LR	10	65.79	20.0
LR	50	43.31	86.0
Linear SVM	3	43.28	38.0
Linear SVM	10	23.35	562.8
RBF SVM	3	84.00	41.4
RBF SVM	10	74.62	655.3
Poly SVM	3	66.73	37.7
Poly SVM	10	73.88	606.7
Sigmoid SVM	3	32.47	43.4
Sigmoid SVM	10	13.44	691.1

Logistic-regression test accuracies are 78.58%, 66.63% and 43.26% for 3, 10 and 50 classes. The SVM ledger contains validation scores only. Its fixed iteration limit makes these settings a reproduction comparison rather than a claim that each kernel has been fully optimized.

4 2 CNN results

Table 6 All 18 CNN validation accuracies

Configuration	K=3	K=10	K=50
V1 raw	90.75	88.77	81.21
V2 raw	91.72	88.50	75.13
V1 binary	89.57	86.56	76.83
V2 binary	90.07	85.06	70.43
V3 binary	88.75	84.12	65.14
V4 binary	89.20	84.73	70.45

Raw inputs outperform binarized inputs for both V1 and V2 at every category count. At 50 classes, V1 raw exceeds V1 binarized by 4.38 percentage points. Removing the second convolution/pooling stage lowers raw-input accuracy to 75.13% for V2. The 50-class CNN V2 test accuracy recorded in the original experiment ledger is also 75.13%; separate error counts show that the equal rounded validation and test percentages are coincidental.

4 3 Transfer learning results

Table 7 Three-class transfer learning results

Network	Val %	Test %	Time s
VGG19	95.35	94.80	206.8
MobileNet	94.58	93.85	124.0
ResNet50	95.35	95.22	169.8
InceptionV3	93.05	92.67	1351.0

Time in Table 7 includes one feature-extraction pass plus head training, and Inception stage-two training where applicable. It is not comparable to the reference paper's seconds-per-epoch figures. The transfer networks were not evaluated at 10 or 50 classes, so their three-class scores cannot be interpreted as superior 50-class performance.

4 4 Independent Mac verification

The assembled Mac environment uses Python 3.12, TensorFlow 2.21.0, Keras 3.15.1 and XGBoost 3.4.1. Native TensorFlow inference correctly classifies 81,071 of 100,000 50-class test drawings with CNN V1 (81.071%). The final XGBoost models classify 5,448/6,000, 17,958/20,000 and 77,886/100,000 correctly: 90.80%, 89.79% and 77.886%. These independent XGBoost scores match the recorded rounded values.

All 18 supplied CNN checkpoints load and produce finite probability outputs. The 10 code cells in the demonstration notebook execute successfully on CPU. Full transfer-learning accuracy and the historical multi-hour training runs were not repeated on the Mac. Model loading and notebook execution are evidence of functionality, not evidence of arbitrary-drawing accuracy.

5 Reproduction analysis

The original report [1] is used as historical context, while project artifacts [3,8] provide the implementation and current results. Agreement is numerical, not proof of identical training trajectories. Different hardware, libraries, implementation details and random variation limit exact replication claims.

Table 8 Selected reference and project validation scores

Model and setting	Ref %	Project %
LR, 3 classes	79.51	79.27
LR, 10 classes	64.64	65.79
LR, 50 classes	43.89	43.31
CNN V1 raw, 50	81.79	81.21
CNN V2 raw, 50	81.83	75.13
VGG19, 3	95.58	95.35
ResNet50, 3	62.72	95.35
InceptionV3, 3	45.72	93.05

5 1 CNN version dispatch

Inspection of the reference repository's cnn.py [9] finds independent version checks followed by an else attached only to version 5. For versions 2-4, that control flow overwrites the chosen model with V1. The bug is directly observable in the code. It plausibly explains why the reported raw V1 and V2 results are close and why the corrected V2 implementation is lower. Which architecture produced every published row remains unverified; the code finding does not justify asserting that all published labels are known to be wrong.

5 2 Data size and SVM convergence

The project tests the hypothesis that more data under the same 500-iteration cap changes the SVM result. Moving from 20,000 to 70,000 records per class lowers the 3-class RBF validation score from 84.00% to 75.03%, and the 10-class score from 74.62% to 64.34%. This supports a convergence-related explanation, but does not eliminate the remaining difference from the historical result.

Table 9 SVM validation at two data sizes

Kernel	K	20k/class	70k/class
Linear	3	43.28	41.49
Linear	10	23.35	22.96
RBF	3	84.00	75.03
RBF	10	74.62	64.34
Poly	3	66.73	54.22
Poly	10	73.88	56.59
Sigmoid	3	32.47	29.55
Sigmoid	10	13.44	10.51

5 3 Transfer learning differences

VGG19 is close to the reference result, while the three networks containing Batch Normalization are substantially higher. Frozen BatchNorm training/inference behavior is a plausible framework-related factor; modern Keras documents its special handling [6]. However, no controlled replay of the old software environment was performed. The explanation remains a hypothesis. The lower CNN V3 binary scores at 10 and 50 classes also remain unexplained; seed and library effects are candidates, not established causes.

6 XGBoost tuning and efficiency

The initial three-class search completed 45 trials; the initial ten-class search completed 31. An extended wider ten-class search contains 32 trials. The final models use 1,301, 2,644 and 2,193 boosting rounds for 3, 10 and 50 classes. At 50 classes, the selected time-capped model was continued from round 1,168, then stopped by validation log-loss. Table 10 separates the early and extended results.

Table 10 XGBoost early and extended test results

K	Early %	Final %	Final rounds
3	90.80	90.80	1,301
10	89.69	89.79	2,644
50	77.44	77.89	2,193

The selected 50-class CNN V1 fit takes 431.43 s versus 7,191.10 s for the continued XGBoost model, a 16.67-fold difference on the recorded hardware. That is a selected-fit comparison, not total search cost. The full tuning effort and feature-extraction costs must be considered when assessing compute efficiency. Original SVM and CNN jobs also sometimes ran concurrently, affecting timings.

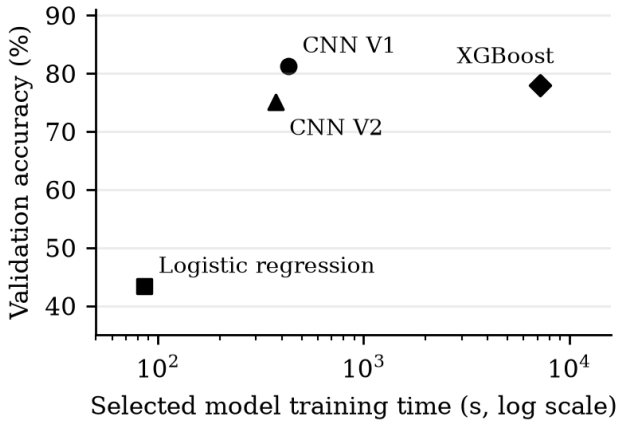


Figure 2 Validation accuracy versus selected fit time on 50 classes. XGBoost search cost is additional. Hardware and algorithms differ; this is not a controlled throughput benchmark.

Extended XGBoost training gains 0.10 test points on 10 classes and 0.45 on 50 classes. These small one-seed gains do not establish statistical significance or a universal 78% ceiling. Different features, data augmentation, search spaces or repeated seeds may change the conclusion.

7 Live inference and rendering shift

A local drawing canvas compares CNN V1 and the final 50-class XGBoost model on the same 28 by 28 image. Prediction runs after each completed stroke, or on request. This is live inference with previously trained weights; it is not live training. A notebook separately demonstrates fast logistic-regression training, saved experiment curves and model predictions.

7 1 Failure investigation

The first canvas used a brush width of 8 pixels on a 400-pixel canvas, cropped the ink, padded the bounding box by a factor of 1.2 and downsampled to 28 by 28. Users observed repeated XGBoost guesses such as stitches. Direct inference and live HTTP outputs agreed, so request serialization was not the cause. Official QuickDraw rasterization uses a specified line diameter, centering and scaling [7]; our canvas produced thinner strokes and a different occupied size.

7 2 Controlled rendering protocol

For each of the 50 categories, the first eight unfiltered simplified vector records were streamed from the official dataset, yielding 400 source drawings. Each was rendered with five brush widths (2, 4, 8, 16, 24 pixels), plus smaller, larger and shifted variants: 3,200 paired cases per preprocessing mode. Another 150 requests per app version checked all 50 categories at three widths through the real prediction endpoint.

These specimens can overlap the original training source. They test rendering sensitivity and must not be presented as independent held-out accuracy. The first records were not filtered for recognition or prediction success. Physical mouse drawing could not be automated in the available browser environment; endpoint and synthetic-rendering checks do not replace a prospective user study.

7 3 Optional input adaptation

The correction estimates line width from a distance transform of the high-resolution ink mask. Ridge distances give an approximate diameter; circular dilation or erosion moves it toward 0.06 times the drawing extent. Padding is reduced to a factor of 1.12. Both models receive the same transformed pixels. A wider default brush of 20 improves usability; a visible checkbox preserves the original preprocessing mode.

Table 11 Source-stroke diagnostic accuracy at width 8

Mode	CNN %	XGBoost %
Original canvas	21.50	4.75
Adapted canvas	83.75	82.00

The large change identifies a practical distribution mismatch, not newly learned model capability. No model weights or historical benchmark scores were changed. The report retains incorrect cases rather than selecting only successful drawing examples.

7 4 Independent bitmap control

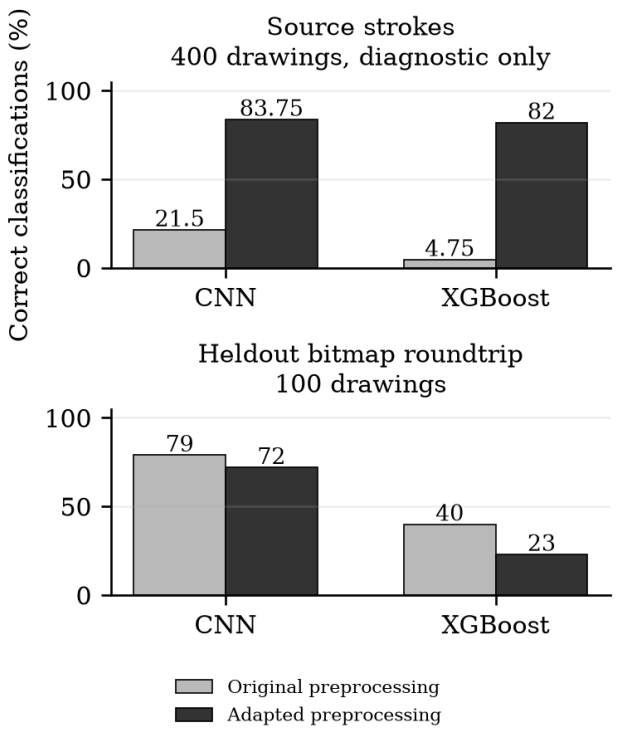


Figure 3 Adaptation has opposite effects in two input regimes. Top: source vector specimens. Bottom: a separate held-out bitmap roundtrip. Each panel compares the same drawings under two preprocessing modes.

An independent audit samples two examples per category from the saved test split, with seed 20261021. On these 100 native 28 by 28 bitmaps, CNN and XGBoost achieve 79% and 75%. After rendering the same bitmaps onto a large white canvas and applying the original preprocessing, the scores are 79% and 40%; XGBoost predicts stitches for 32 drawings.

Table 12 Held-out bitmap control with the same 100 items

Input	CNN %	XGB %
Native test pixels	79	75
Canvas roundtrip	79	40
Adapted roundtrip	72	23

The adaptation helps clean vector-style strokes but worsens this already-rasterized bitmap regime. It is therefore optional, not a universal correction. XGBoost's strong sensitivity to exact pixel appearance is consistent with its threshold-based representation. CNN retains more performance in the unadapted bitmap roundtrip, but it too remains vulnerable to drawing style and preprocessing changes.

A 25-26-pixel occupied-size sweep reduces the XGBoost stitches collapse in bitmap controls, but geometry alone does not recover the native score. Future evaluation should use genuinely new human drawings, record pen width and canvas resolution, and separate calibration drawings from a locked evaluation set.

8 Discussion and limitations

Three conclusions follow from the measured evidence. First, among the tested 50-class configurations, a compact raw-input CNN offers the best validation accuracy and a substantially shorter selected fit than tuned raw-pixel XGBoost. Second, modern transfer learning performs strongly on three classes, but those results do not establish performance on the larger vocabularies. Third, the live-input experiment demonstrates that a valid checkpoint and an accurate saved test result are insufficient to guarantee reliable freehand predictions.

Most configurations use one seed. There are no confidence intervals or multi-seed significance tests. XGBoost receives a substantial tuning budget whereas reproduced baselines keep fixed settings; comparisons therefore have unequal optimization effort. Fit times were measured on different devices and some concurrent jobs. The current report does not claim a hardware-independent speed ranking.

Data splits are disjoint by selected row index, but exact duplicate bitmaps may remain across splits. The verifier permits small overlaps; an audit found one identical belt bitmap shared between training and test in the three-class archive. A stricter content-deduplicated benchmark would be preferable before making strong generalization claims. The first-20,000 selection also differs from a random sample over every available drawing.

Validation selects settings and boosting rounds. Test values exist for the early and extended XGBoost phases and for several model families. They should be read as recorded final-phase measurements, not evidence that test access occurred once for the entire study. Follow-up work should lock one final test evaluation after model and input-adaptation choices are complete.

8 1 Reproducibility and runtime

The project retains class lists, fixed split arrays, 46 run records, per-run JSONs, curves, confusion matrices and supplied trained weights. Mac dependencies are isolated in a Python environment with a recorded version lock. TensorFlow and XGBoost use CPU inference; NVIDIA CUDA packages are not required. The original Linux package list is preserved separately. The supplied source snapshot accompanies the experiment records and trained-model bundles [3].

Fresh Mac checks confirm the main CNN and all three final XGBoost test scores, finite outputs from every CNN checkpoint, and successful execution of the ten demo cells. Transfer-learning heads are supplied, but their accuracy was not independently rerun on the Mac. Automated endpoint checks cover blank input, examples, clearing, adaptation on/off and direct-versus-web agreement [8].

9 Conclusion

The recorded experiments support compact CNNs for efficient recognition of the chosen doodle categories. XGBoost is a meaningful non-neural extension and approaches 78% validation accuracy on 50 classes, but raw-pixel tree predictions can be fragile under canvas rendering changes. The optional stroke adaptation addresses one important freehand regime while failing on another. Reliable deployment therefore requires evaluation of the complete drawing-to-prediction pipeline, with input modes and limitations made explicit.

10 Future work

The next study should collect new freehand drawings from multiple people and input devices, freeze preprocessing before testing, and evaluate both top-1 and top-5 accuracy by category and pen width. Multiple seeds, content-deduplicated splits, confidence intervals and balanced tuning budgets would strengthen comparisons. Feature-based XGBoost using frozen visual embeddings is a useful extension to test, rather than assume. Unknown-class rejection and calibration are also needed before interpreting scores as dependable confidence.

References

[1] N. M. Phu, C. Xiao and J. Muindi. Drawing A New Way To Search. Stanford CS229 project report, 2018. cs229.stanford.edu/proj2018/report/33.pdf

[2] Google Creative Lab. The Quick Draw Dataset. Dataset documentation and format specification. github.com/googlecreativelab/quickdraw-dataset. Accessed 22 October 2026.

[3] Supplied project source code and historical experiment records, 2026. Model checkpoints and data bundles accompany the implementation.

[4] T. Chen and C. Guestrin. XGBoost A Scalable Tree Boosting System. KDD, 2016. doi.org/10.1145/2939672.2939785. arxiv.org/abs/1603.02754.

[5] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama. Optuna A Next-generation Hyperparameter Optimization Framework. KDD, 2019. arxiv.org/abs/1907.10902.

[6] Keras. Transfer learning and fine-tuning. Official guide, section on BatchNormalization. keras.io/guides/transfer_learning/. Accessed 22 October 2026.

[7] Google Creative Lab. QuickDraw dataset issue 19. Contributor rasterization implementation, 2018. github.com/googlecreativelab/quickdraw-dataset/issues/19. Accessed 22 October 2026.

[8] Project artifact snapshot, 22 October 2026. results/all_results.csv, results/runs/, verification/demo_execution.json, verification/canvas_study/ and verification/independent_canvas_audit/. Local reproducibility evidence; not a separate published benchmark.

[9] jervisfm. cs229-project. Source implementation associated with [1]. github.com/jervisfm/cs229-project. Archived local source inspected 22 October 2026.

Appendix A Category vocabulary

The 50-class order is: squirrel, paint can, stitches, lighter, panda, hockey stick, kangaroo, rain, sea turtle, aircraft carrier, beard, chair, hammer, hedgehog, spreadsheet, camel, nose, peas, wristwatch, banana, belt, eraser, harp, shark, skull, face, castle, drill, lobster, snowflake, speedboat, bee, fence, floor lamp, hat, leaf, ambulance, pond, sweater, grapes, laptop, screwdriver, broccoli, finger, teddy-bear, umbrella, trumpet, snake, star and The Mona Lisa.